Система контроля взаимодействия ИТ Школы Ростелекома с вузами

CRM для кейса «ИТ Школа» хакатона «Лидеры цифровой трансформации 2026». Визуализирует многоэтапный процесс взаимодействия с вузами-партнёрами, ведёт учёт договоров, задач и комментариев по каждому взаимодействию.



Прототип

Рабочий прототип: https://crm-rtk.pixel-minds.ru

Демо-доступ:

Роль	Логин	Пароль
Администратор	`admin@rtk.ru`	`admin123`
Руководитель	`manager@rtk.ru`	`manager123`
KAM	`kam@rtk.ru`	`kam123`

Что реализовано

- **Kanban-доска** взаимодействий с перемещением карточек между этапами (`@dnd-kit`), поиском по вузу и фильтром по продукту.
- **Карточка взаимодействия**: сведения, договор, задачи со сроками, комментарии, смена этапа.
- **Справочники**: вузы, ИТ-продукты, ИТ-направления.
- **Настройка воркфлоу**: добавление этапов, переименование, цвет колонки, порядок, включение/выключение.
- **Отчёт**: ключевые показатели и распределение взаимодействий по этапам.
- **Роли**: менеджер и администратор (доступ к справочникам и настройкам — только у администратора).
- **Адаптив**: на мобильной доска показывается одна колонка с выбором этапа.
- **Интеграция с LMS и CMS**: контракт обмена, предпросмотр пакета (dry-run) и идемпотентный импорт по `contract_number` (mock-адаптер).
- **ИИ-сводка**: краткая выжимка по карточке через GigaChat.
- **Вход через Keycloak**: Authorization Code Flow с PKCE; локальный JWT-вход остаётся запасным вариантом.

Ограничение бизнес-модели

У одного вуза может быть **не более 2 активных** взаимодействий — проверяется на backend при создании.

Стек

Слой	Технологии
Backend	Python 3.11, FastAPI, SQLAlchemy 2 (async), JWT + bcrypt
Frontend	React 18, TypeScript, Vite, Ant Design 5, Zustand, @dnd-kit
БД	PostgreSQL 16 (контейнер или Yandex Managed PostgreSQL) или SQLite для локальной разработки
Кэш	Redis / KeyDB для агрегатов отчётов
Хранилище файлов	Yandex Object Storage или MinIO (S3-совместимое)
Деплой	основной контур — Yandex Cloud (BM + nginx); альтернативный — Render + Vercel + Supabase

Бизнес-модель

```
University  ─┐
             ├─► Interaction ─► WorkflowStage
ITProduct   ─┘         │
ITDirection ── ITProduct       ├── Action (задача со сроком)
                               └── Comment
```

`University`, `ITProduct`, `ITDirection` — справочники. `Interaction` — главная сущность, связка «вуз + продукт», перемещается по этапам.

Структура репозитория

```
backend/          FastAPI-приложение (пакет backend)
  core/config.py    настройки из окружения
  db/               база и сессия
  models/           модели и перечисления
  schemas/          Pydantic-схемы
  api/              роутеры: auth, universities, directories,
                    stages, interactions, reports
  seed.py           справочники + демо-данные
  certs/            корневой сертификат НУЦ Минцифры для TLS к GigaChat
  start.sh          старт без Docker (Render, Start Command)
frontend/         React-приложение (Vite)
  src/api/          axios-клиент и описание эндпоинтов
  src/pages/        BoardPage, ReportPage, DirectoryPage,
                    SettingsPage, LoginPage
  src/components/   MainLayout, kanban, interaction
infra/yandex-cloud/ прод-стек для BM: setup-vm.sh, docker-compose, nginx
render.yaml       Blueprint для контура Render + Vercel + Supabase
docs/             DEPLOYMENT.md, STACK.md, USER_GUIDE.md,
                  architecture/, security/, pitch.md
```

Быстрый старт через Docker Compose

Всё приложение поднимается одной командой. Нужен только Docker с плагином Compose — Python и Node на машине не требуются.

```
git clone https://github.com/ForgeLabHub/ForgeRTKCRM.git rtk-crm
cd rtk-crm
cp .env.example .env
# заполнить .env — см. ниже
docker compose up -d --build
```

Обязательные значения в `.env`:

Переменная	Как получить
`SECRET_KEY`	`openssl rand -hex 32`
`MINIO_ROOT_PASSWORD`	придумать (пароль хранилища файлов)
`KEYCLOAK_ADMIN_PASSWORD`	придумать (админ Keycloak)
`DATABASE_URL`	по умолчанию SQLite — для контейнера оставить как есть

Остальные переменные имеют рабочие значения по умолчанию. Без ключа GigaChat приложение запускается: вкладка «Сводка» сообщит, что сервис не настроен.

Проверка готовности:

```
make doctor        # окружение до запуска: инструменты, .env, порты
docker compose ps
curl http://localhost:8000/healthz
curl http://localhost:8000/readyz
make smoke-test    # healthz, readyz, /docs, /openapi.json, фронтенд
```

Команды разработки

Команда	Что делает
`make doctor`	проверка инструментов, версий, `.env`, портов; ничего не меняет
`make test`	pytest на SQLite, внешняя СУБД не нужна
`make lint typecheck`	ruff + mypy
`make alembic-check`	модели и миграции согласованы
`make compose-up` / `make compose-down`	стек; `SERVICES="backend frontend"` для минимума
`make docs-check`	полнота документации и спецификаций модулей
`make verify`	всё перечисленное сразу

Матрица «утверждение → команда → CI»: docs/verification/matrix.md. Точка входа в документацию: docs/index.md.

Адреса после запуска:

Сервис	Адрес
Frontend	http://localhost:3000
Backend / Swagger UI	http://localhost:8000/docs
Keycloak	http://localhost:8080
MinIO Console	http://localhost:9001
Nginx	http://localhost

> **Swagger UI и `/docs` на домене.** Swagger UI отдаёт сам backend по `/docs` — локально это `http://localhost:8000/docs`. На публичном домене путь `/docs` занят статической документацией фронтенда (`frontend/public/docs/`), поэтому Swagger UI живёт под тем же адресом, что и API: `https://crm-rtk.pixel-minds.ru` + `/api` (см. `frontend/nginx.conf`).

Схема БД и справочники создаются самим приложением при старте (lifespan): миграции и seed вручную запускать не нужно. Демо-данные включаются переменной `SEED_DEMO_DATA` (по умолчанию `true`).

Остановить: `docker compose down`. Удалить вместе с данными: `docker compose down -v`.

Альтернатива без Docker — раздел «Локальный запуск» ниже. Готовый интерактивный скрипт с меню: `./run.sh start` (или `run.bat start` на Windows).

Локальный запуск

Backend

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r backend\requirements.txt
# создать .env в корне репозитория:
# DATABASE_URL=sqlite+aiosqlite:///./dev.db
python scripts\run_dev.py
```

`Settings` читает `.env` из текущего каталога, поэтому файл кладётся в корень репозитория (рядом с `README.md`), а не в `backend/`.

Запускать нужно через `scripts/run_dev.py`: приложение живёт как пакет `app` (в контейнере — `/app`, в репозитории — `backend/`), поэтому `uvicorn backend.main:app` из корня падает с `ModuleNotFoundError: app`. Скрипт делает пакет импортируемым и поднимает uvicorn с автоперезагрузкой.

Swagger: http://127.0.0.1:8000/docs

Frontend

```
cd frontend
npm install
npm run dev
```

http://127.0.0.1:5173 — dev-сервер проксирует `/api` на `localhost:8000`, поэтому `VITE_API_URL` в разработке не нужен.

Демо-доступ

Роль	Email	Пароль
Администратор	`admin@rtk.ru`	`admin123`
Менеджер	`manager@rtk.ru`	`manager123`
КАМ	`kam@rtk.ru`	`kam123`

Вход и аутентификация

Система поддерживает два режима, переключаемых `AUTH_MODE` на backend и `VITE_AUTH_MODE` на frontend.

Локальный JWT (`jwt`, по умолчанию) — вход по email и паролю из формы. Подходит для демонстрации и запуска без Keycloak.

Keycloak (`keycloak`) — кнопка **Войти через Keycloak** на странице входа. Используется Authorization Code Flow с PKCE (`S256`): публичный клиент без client secret, код авторизации бесполезен для перехватчика. Backend проверяет RS256-токен по JWKS realm и берёт роли из claims.

Если Keycloak недоступен, страница входа не пустует: фронтенд проверяет доступность сервера и показывает форму email/пароль как запасной путь.

Подробности — в docs/KEYCLOAK.md.

Деплой

Поддерживаются два контура; код один и тот же, различия — только в переменных окружения.

Yandex Cloud (основной). Одна BM, Docker Compose из пяти контейнеров (PostgreSQL, KeyDB, backend, frontend, nginx). Данные размещаются в РФ — это целевой контур по 152-ФЗ и приказу ФСТЭК № 117.

```
docker compose -f infra/yandex-cloud/docker-compose.prod.yml --env-file .env up -d
```

Пошагово — в infra/yandex-cloud/README.md и docs/DEPLOYMENT.md.

Render + Vercel + Supabase (альтернативный). Backend — контейнер на Render, frontend — статика на Vercel, база — Supabase PostgreSQL. Подходит для быстрого публичного стенда без своей инфраструктуры.

Документация

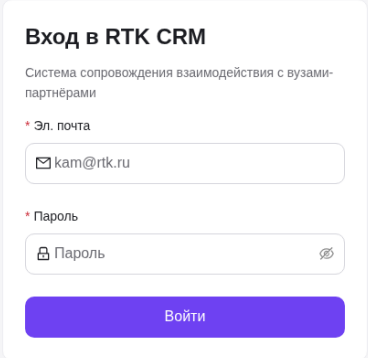
- rtk-crm-documentation.pdf — единый PDF со всей документацией
- RTK-CRM-LCT2026.pptx — презентация, 11 слайдов по шаблону ЛЦТ
- RTK-CRM-LCT2026.pdf — та же презентация в PDF
- DEPLOYMENT.md — развёртывание
- ARCHITECTURE.md — архитектура
- KEYCLOAK.md — вход через Keycloak и локальный JWT
- AI.md — сводка по взаимодействию (GigaChat)
- USER_GUIDE.md — руководство пользователя

- ADMIN_GUIDE.md — руководство администратора
- SECURITY.md — безопасность, 152-ФЗ и privacy
- pitch.md — презентационный сценарий
- qa-jury.md — вопросы жюри и ответы

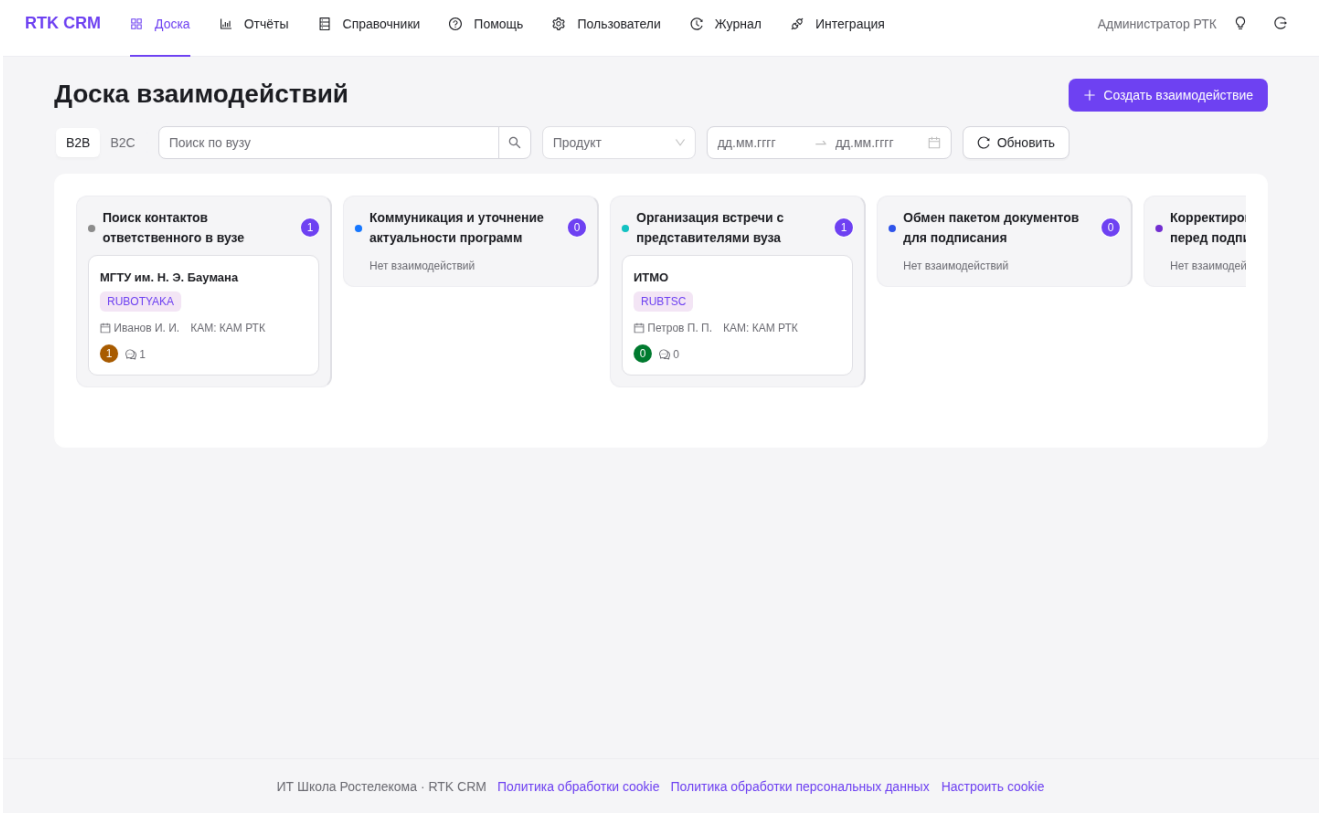
Интерфейс

Скриншоты актуальной сборки лежат в docs/images.

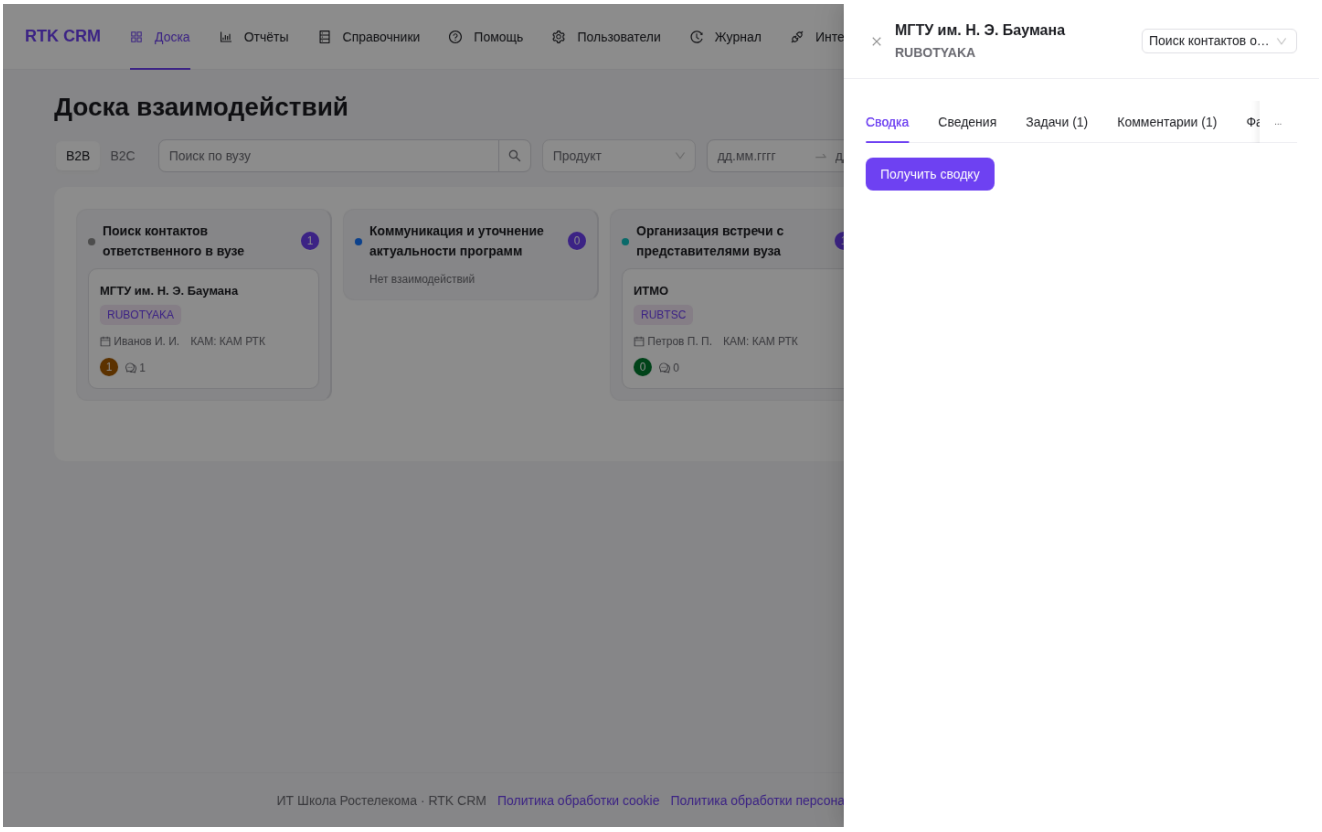
Вход в систему



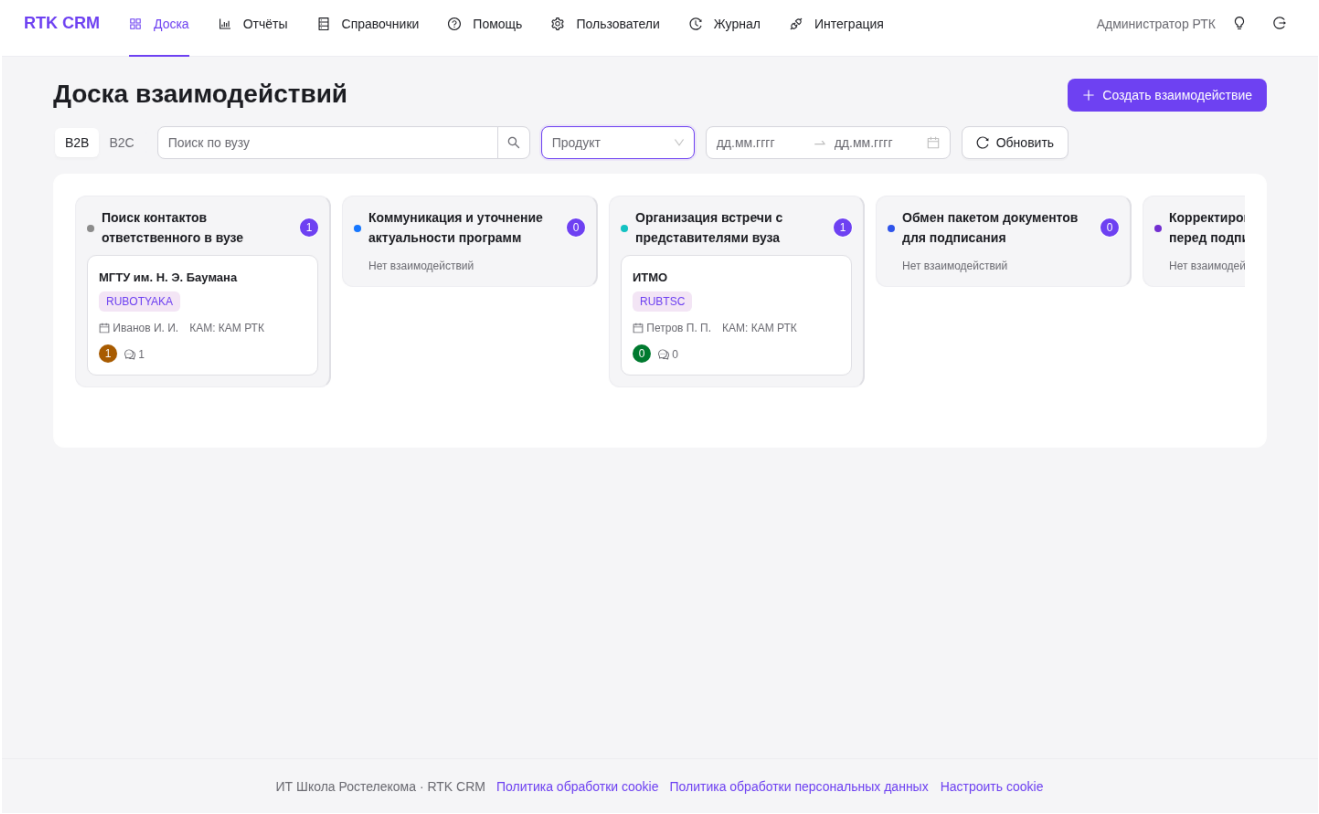
Доска взаимодействий (Kanban)



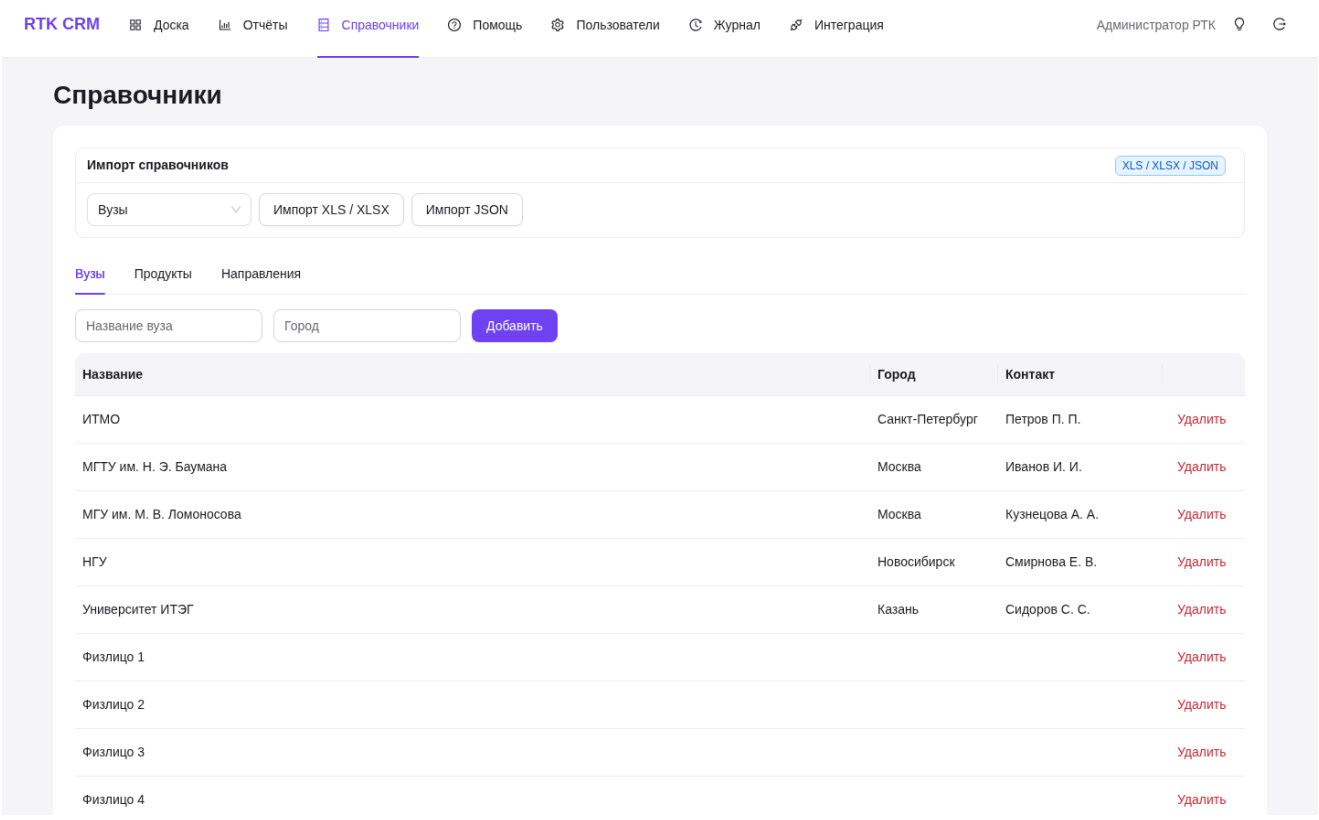
Карточка вуза



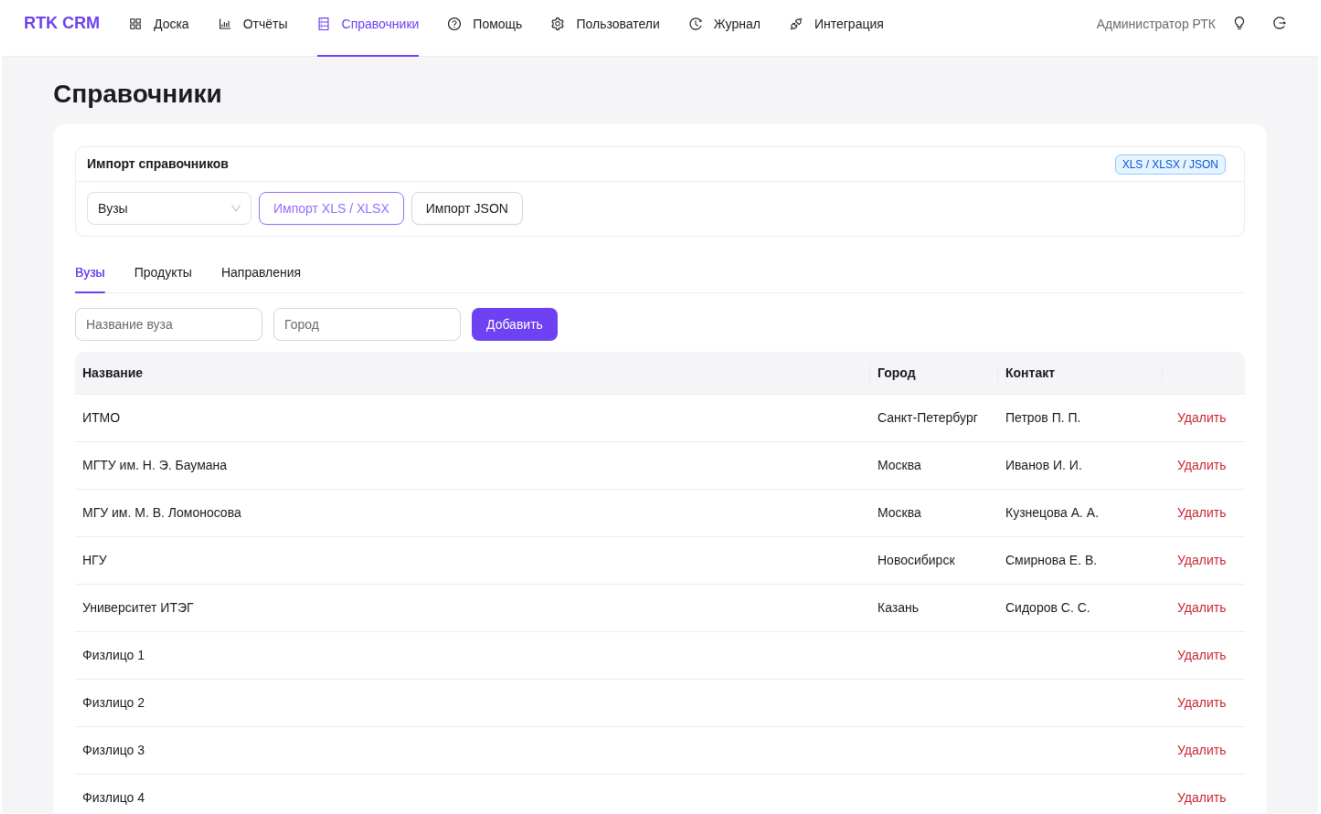
Фильтры на доске



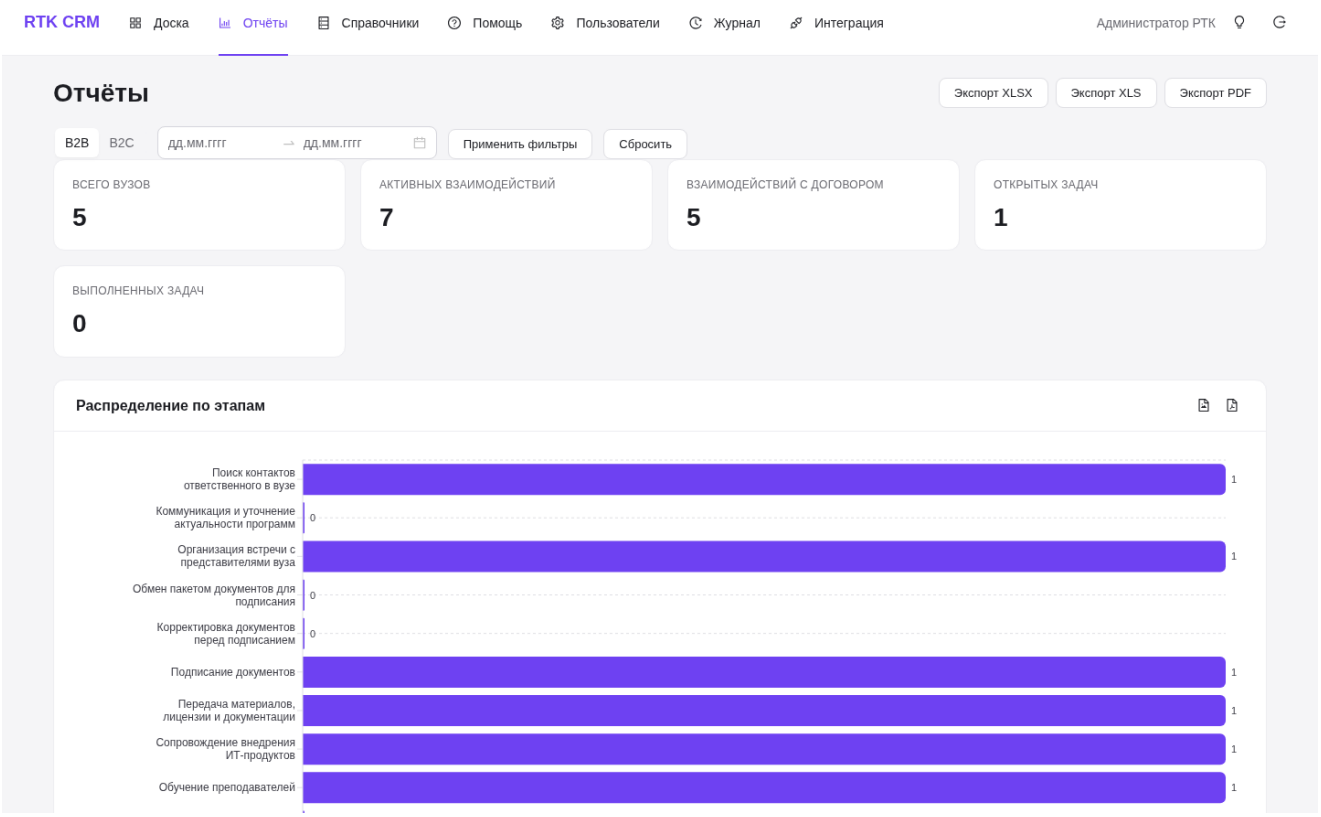
Справочники



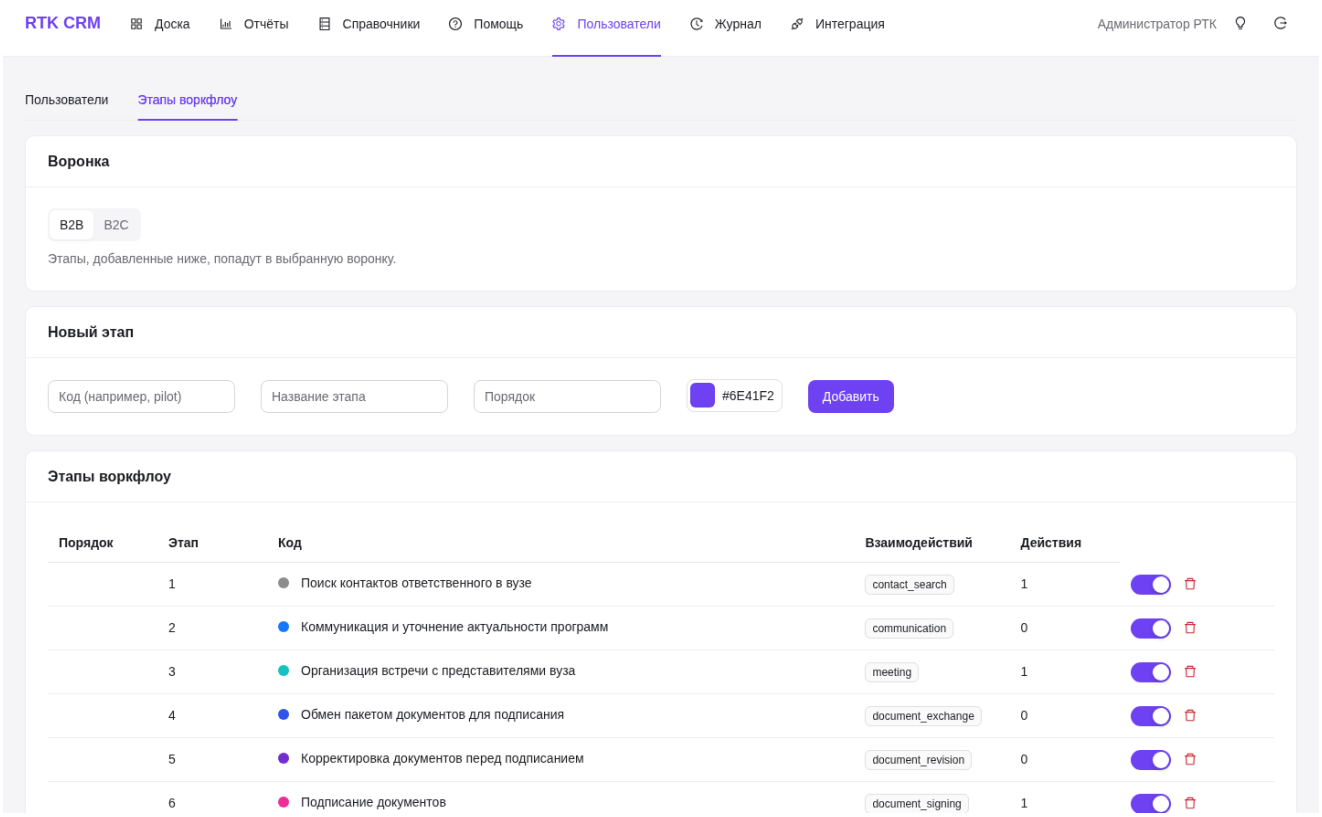
Импорт XLSX и сопоставление полей



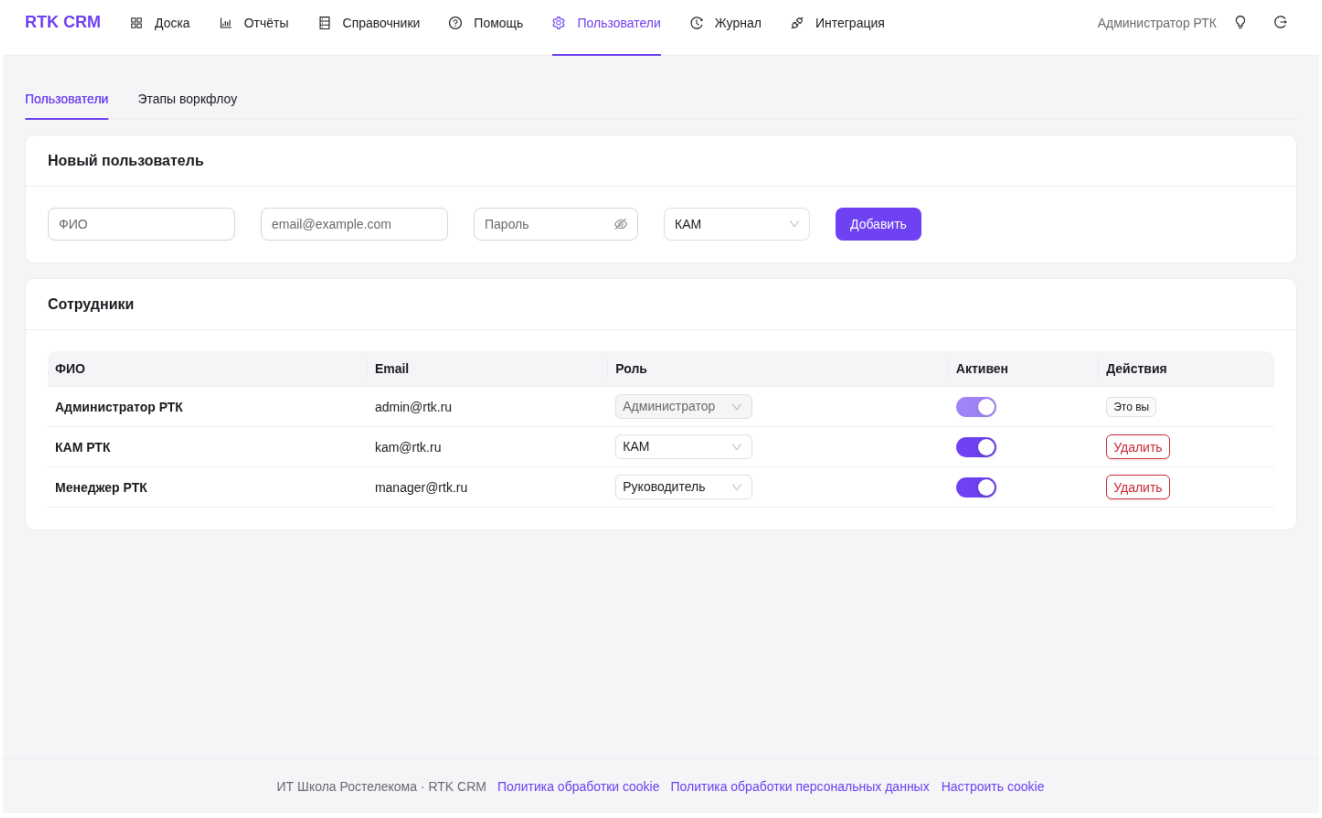
Отчёты с экспортом диаграмм в PNG/PDF



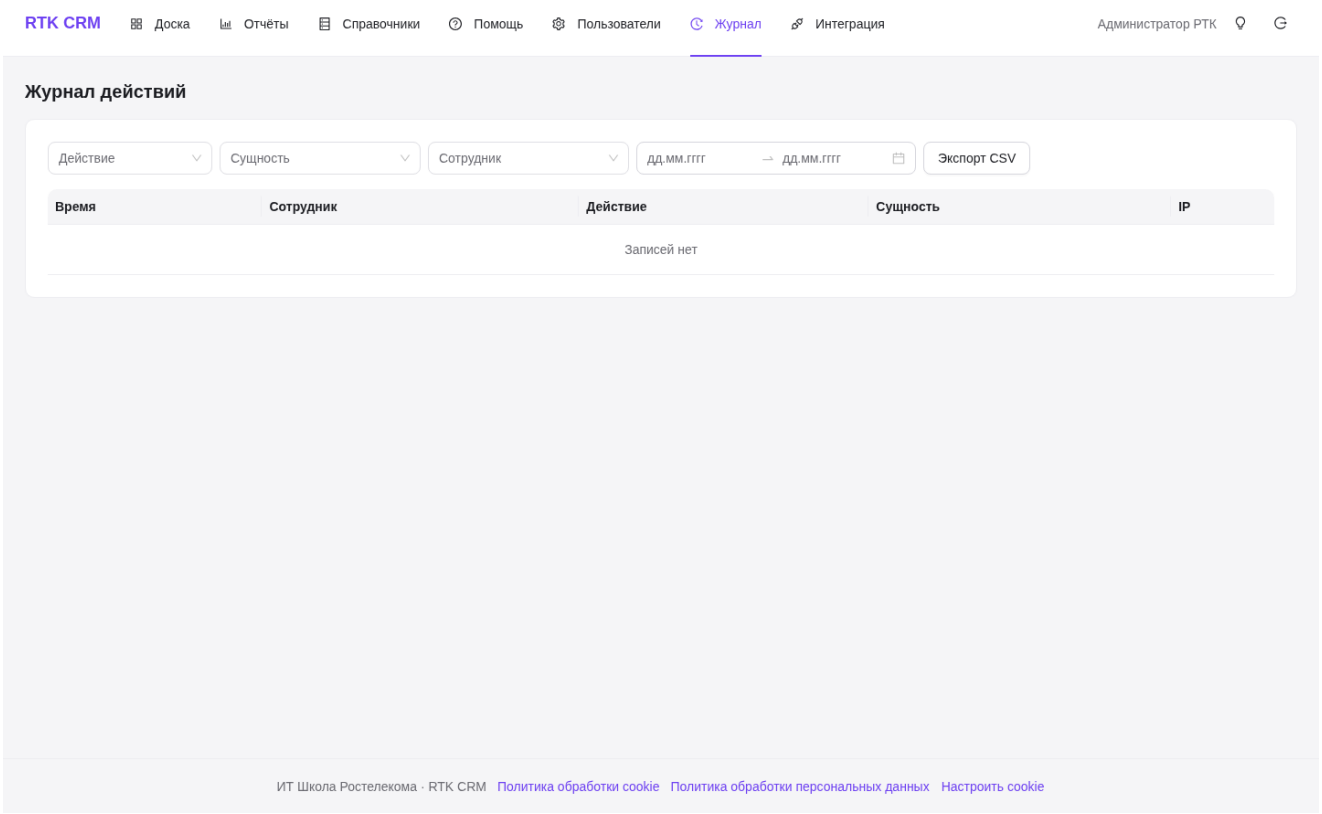
Конструктор воркфлоу



Пользователи (администратор)

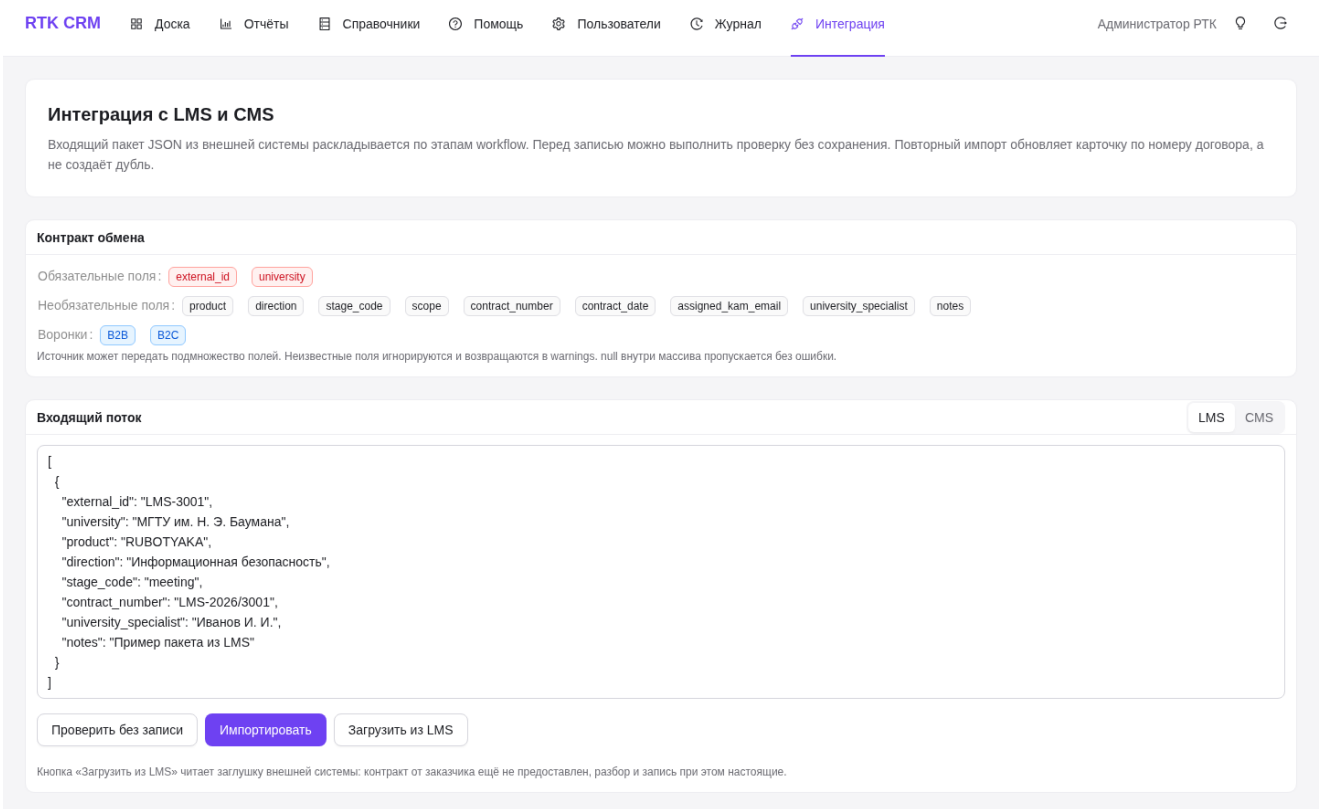


Журнал аудита (администратор)

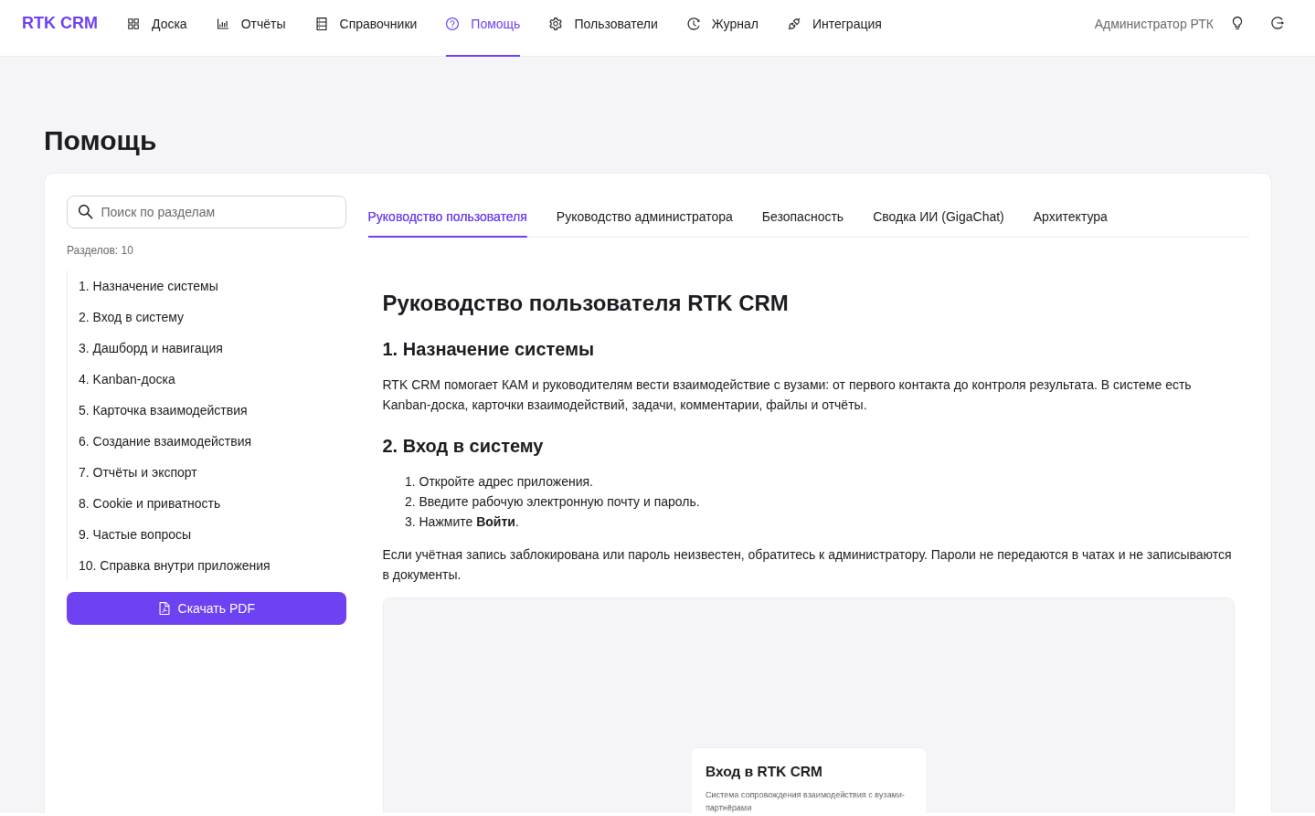


Интеграция с LMS и CMS

Вкладка описывает обмен с внешними системами заказчика: контракт обмена, предпросмотр пакета (dry-run) и идемпотентный импорт по `contract_number`.



Встроенная документация `/help`



Production

- Основной контур: Yandex Cloud, `https://<домен-ВМ>` (см. docs/DEPLOYMENT.md)
- Альтернативный контур: Render + Vercel + Supabase

Нагрузочная проверка

```
locust -f tests/load/locustfile.py RTKUser \
  --headless -u 50 -r 5 -t 60s \
  --host=http://localhost:8000 \
  --html=tests/load/report.html
```

Для отдельной проверки десяти параллельных отчётов используйте сценарий `ReportsUser`; подробности находятся в tests/load/README.md.

Лицензия

Проект распространяется по лицензии MIT.

Управление проектом: `forge.sh` и `forge.bat`

Один скрипт на Unix-подобные системы (`forge.sh`) и один на Windows (`forge.bat`). Интерфейс одинаковый: те же команды, та же логика, одна реализация на каждой платформе — без bash-скриптов внутри Windows.

```
./forge.sh              # интерактивное меню
./forge.sh doctor       # проверить окружение, ничего не меняя
./forge.sh install      # поставить зависимости
./forge.sh start        # запустить в фоне, дождаться готовности, открыть браузер
./forge.sh status       # что запущено и отвечает ли порт
./forge.sh logs api -f  # логи в реальном времени
./forge.sh test         # тесты
./forge.sh stop         # остановить
```

```
forge.bat
forge.bat start
forge.bat status
```

Что умеет

Команда	Действие
`status`	состояние по портам и HTTP, а не «на глаз»
`start [--fg]`	фон (PID-файлы в `.run/`) или передний план
`stop`	останавливает дерево процессов целиком, порт освобождается
`restart`, `logs [имя] [-f]`, `open`	обычные операции
`doctor`	инструменты, версии, `.env`, `.env` в git, занятость порта
`install`, `test`, `lint`, `build`, `verify`	через `make`, если он есть
`migrate`	применить миграции

| `db migrate\|backup\|restore` | база: миграции, копия, восстановление | | `docker up\|down\|logs\|reset` | контейнеры (`down` данные не удаляет) | | `clean [--hard]` | артефакты и логи; `--hard` ещё `.venv` и `node_modules` |

Правила, заложенные в скрипт

- **Ничего не меняет без команды.** `doctor` и `status` только читают.
- **Данные не удаляются.** `stop` и `docker down` сохраняют базу; удаление — только `--hard` или явное подтверждение в `docker reset`.
- **Предполётная проверка.** Если нет `go`/python или занят порт — скрипт говорит об этом сразу, а не через две минуты таймаута.
- **Не висит после падения.** Если процесс умер, запуск прекращается и показывает последние строки лога.
- **Venv главнее системы.** Если есть `.venv`, им пользуются `make`, `pytest` и uvicorn — чтобы `install` и `test` не расходились.

Текущий способ запуска без скрипта: `python scripts/run_dev.py` — адрес `http://localhost:8000/docs`.

Конфигурация: `.env`

Скрипт читает `.env` **собственным парсером**, а не через `source`: файл приходит из репозитория, а выполнять из него произвольный код нельзя. Поддержаны комментарии `#`, пустые строки, префикс `export`, одинарные и двойные кавычки, CRLF, пробелы вокруг `=` и комментарий после значения.

Приоритет значений: **переменная окружения → `.env` → встроенное значение**. Поэтому `PORT=9000 ./forge.sh start` работает независимо от файла.

Команда	Действие
`env` / `env show`	переменные, источник каждого значения и замаскированные секреты
`env init [--force]`	создать `.env` из `.env.example`: заглушки секретов заменяются случайными, права `600`, прежний файл — в `.env.bak`
`env set KEY=VALUE`	записать или заменить одну переменную (атомарно, через временный файл)
`env check`	сверка с `.env.example`: пропавшие переменные, пустые и «примерные» секреты, `.env` в `.gitignore`
`env edit`	открыть файл в `$EDITOR` (fallback: nano → vim → vi)
`env path`, `env reload`	путь к файлу и перечитывание без перезапуска

Секреты в выводе маскируются: `ADMIN_CODE=ch…de (31 симв.)`, пароль внутри строки подключения — `postgres://user:***@host`. Имена вида `*_MINUTES`, `*_URL`, `*_PORT` секретами не считаются: маскировать число минут бессмысленно.

Путь к файлу переопределяется: `FORGE_ENV_FILE=/etc/app.env ./forge.sh status`. В Windows-версии те же команды: `forge.bat env init`, `forge.bat env check`. Ограничение `cmd.exe`: значения, содержащие `%` и `!`, не поддерживаются.

Флаги и автодополнение

```
./forge.sh -h | --help            # справка
./forge.sh -V | --version         # версия скрипта, коммит, состояние .env
./forge.sh -q status              # молчать об успехах
./forge.sh --no-color status      # без ANSI (то же делает NO_COLOR=1)
./forge.sh --json status          # машинный вывод для скриптов и мониторинга

eval "$(./forge.sh completion bash)"   # автодополнение для bash
./forge.sh completion zsh > ~/.zfunc/_forge   # для zsh
```

`test`, `lint`, `build` и `verify` запускаются в окружении **без** переменных из `.env`: иначе локальные проверки вели бы себя иначе, чем CI, и тесты вида «без ключа должен быть fallback» перестали бы что-либо проверять.

Запуск в этом проекте

```
./forge.sh install && ./forge.sh start
```